

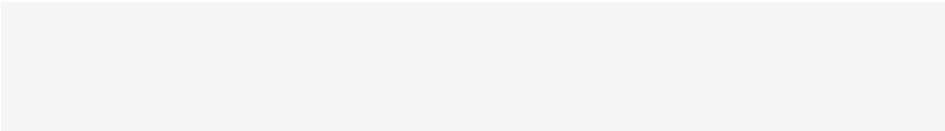
Absolutely. If you're preparing for a **DevOps / Platform / Cloud Engineer** role, Linux commands should become muscle memory—not something you look up every time.

Below is the command set I recommend practicing **daily**, especially for your Docker + Node.js/NestJS + microservices + PostgreSQL/Kafka/Redis environment.

1. File & Directory Commands

Command	Example	Use case
<code>pwd</code>	<code>pwd</code>	Show current directory
<code>ls</code>	<code>ls -lah</code>	List files with permissions, size, hidden files
<code>cd</code>	<code>cd /var/log</code>	Move between directories
<code>mkdir</code>	<code>mkdir -p app/logs</code>	Create directories
<code>touch</code>	<code>touch app.log</code>	Create an empty file
<code>cp</code>	<code>cp app.log backup.log</code>	Copy files
<code>mv</code>	<code>mv app.log logs/</code>	Move/rename files
<code>rm</code>	<code>rm app.log</code>	Delete a file
<code>rm -rf</code>	<code>rm -rf old-build/</code>	Delete directory recursively
<code>file</code>	<code>file app.log</code>	Identify file type
<code>stat</code>	<code>stat app.log</code>	View file metadata
<code>tree</code>	<code>tree -L 2</code>	Visualize directory structure

Daily practice:



```
mkdir -p ~/devops/linux/{app,logs,backup}  
cd ~/devops/linux  
touch app/app.log  
cp app/app.log backup/  
ls -lah  
tree
```

2. Reading Files & Logs

These are **extremely important for DevOps**.

Command	Example	Use case
cat	cat app.log	Read small files
less	less app.log	Read large files
head	head -20 app.log	First 20 lines
tail	tail -20 app.log	Last 20 lines
tail -f	tail -f app.log	Follow live logs
grep	grep "ERROR" app.log	Search text
grep -i	grep -i "error" app.log	Case-insensitive search
grep -r	grep -r "PORT" .	Search recursively
wc	wc -l app.log	Count lines
sort	sort app.log	Sort output
uniq	uniq -c	Count duplicate lines
cut	cut -d: -f1 /etc/passwd	Extract fields
awk	awk '{print \$1}' access.log	Process columns

sed	sed 's/ERROR/WARN/g' app.log	Transform text
-----	---------------------------------	----------------

Real DevOps example

Your NestJS container is producing errors:

```
docker logs banking-api
```

You want only errors:

```
docker logs banking-api | grep -i error
```

Follow errors in real time:

```
docker logs -f banking-api | grep -i error
```

This combination should become second nature.

3. Linux Process Management

When something is consuming CPU/RAM or hanging, these commands are essential.

Command	Example	Use case
ps	ps aux	List processes
ps aux grep	ps aux grep node	Find Node processes
top	top	Live CPU/memory monitoring

htop	htop	Better interactive process monitor
pgrep	pgrep node	Find process PID
kill	kill 1234	Gracefully terminate process
kill -9	kill -9 1234	Force terminate
pkill	pkill node	Kill matching processes
jobs	jobs	View shell jobs
bg	bg %1	Resume job in background
fg	fg %1	Bring job to foreground
nohup	nohup ./app &	Keep process running after logout

Example

Find your Node.js process:

```
ps aux | grep node
```

Example output:

```
ravinder 1234 85.2 ... node dist/main.js
```

Check it:

```
top -p 1234
```

Terminate gracefully:

```
kill 1234
```

Only use:

```
kill -9 1234
```

when a normal `kill` doesn't work.

4. CPU & Memory

Command	Example	Use case
<code>free</code>	<code>free -h</code>	Check RAM
<code>uptime</code>	<code>uptime</code>	Load average + uptime
<code>lscpu</code>	<code>lscpu</code>	CPU information
<code>nproc</code>	<code>nproc</code>	Number of CPUs
<code>vmstat</code>	<code>vmstat 1</code>	CPU/memory/system statistics
<code>top</code>	<code>top</code>	Process resource usage
<code>htop</code>	<code>htop</code>	Interactive monitoring
<code>watch</code>	<code>watch free -h</code>	Repeat command
<code>dmesg</code>	<code>dmesg</code> <code>tail</code>	Kernel messages

Important habit

When a server is slow:

```
uptime
free -h
top
df -h
```

This gives you a quick first diagnosis.

5. Disk & Storage

Very important in production.

Command	Example	Use case
df	df -h	Disk filesystem usage
du	du -sh *	Directory sizes
du	du -sh /var/log/*	Find large logs
lsblk	lsblk	Block devices
mount	mount	Mounted filesystems
find	find /var/log -type f	Find files
find -size	find . -size +1G	Find huge files
find -mtime	find . -mtime +7	Files older than 7 days

Production scenario

Server says:

```
No space left on device
```

Immediately:

```
df -h
```

Then:

```
du -sh /var/*
```

Then:

```
du -sh /var/log/*
```

Find huge files:

```
find /var -type f -size +1G 2>/dev/null
```

This is a **real DevOps troubleshooting workflow**.

6. Permissions & Ownership

Linux permissions are foundational.

Command	Example	Use case
chmod	chmod 755 script.sh	Change permissions
chown	chown user:user app.log	Change owner
chgrp	chgrp developers app.log	Change group
ls -l	ls -l app.log	Check permissions
umask	umask	Default permission mask
id	id	User/group information
whoami	whoami	Current user

Understand:

```
-rwxr-xr--
```

as:

```
owner  group  others
rwx    r-x    r--
```

And know:

```
chmod 755 script.sh
```

means:

```
Owner   = rwx
Group   = r-x
Others  = r-x
```

7. Users & Groups

Command	Example	Use case
whoami	whoami	Current user
id	id ubuntu	User/group information
who	who	Logged-in users
w	w	Logged-in users + activity
passwd	passwd	Change password
su	su - deploy	Switch user
sudo	sudo systemctl restart nginx	Execute as root
useradd	sudo useradd appuser	Create user

<code>usermod</code>	<code>sudo usermod -aG docker appuser</code>	Modify user
----------------------	--	-------------

8. Networking — Must Know

For DevOps, this section is critical.

Command	Example	Use case
<code>ip</code>	<code>ip addr</code>	View IP addresses
<code>ip route</code>	<code>ip route</code>	Routing table
<code>ping</code>	<code>ping google.com</code>	Basic connectivity
<code>curl</code>	<code>curl http://localhost:3000</code>	Test HTTP service
<code>wget</code>	<code>wget https://example.com/file</code>	Download/test HTTP
<code>ss</code>	<code>ss -tulpn</code>	View listening ports
<code>netstat</code>	<code>netstat -tulpn</code>	Older port inspection
<code>dig</code>	<code>dig google.com</code>	DNS lookup
<code>nslookup</code>	<code>nslookup google.com</code>	DNS lookup
<code>traceroute</code>	<code>traceroute google.com</code>	Network path
<code>hostname</code>	<code>hostname</code>	Machine hostname
<code>hostname -I</code>	<code>hostname -I</code>	Machine IP

Extremely important

Suppose your NestJS application should listen on port 3000.

Check:

```
ss -tulpn | grep 3000
```

Test locally:

```
curl http://localhost:3000
```

Test a health endpoint:

```
curl http://localhost:3000/health
```

Check DNS:

```
dig api.example.com
```

9. SSH

You will use SSH constantly.

Command	Example	Use case
ssh	ssh user@server	Connect to server
ssh -p	ssh -p 2222 user@server	Custom SSH port
ssh -i	ssh -i key.pem user@server	SSH with key

scp	scp app.tar user@server:/tmp/	Copy files
rsync	rsync -av app/ user@server:/app/	Efficient synchronization
ssh-keygen	ssh-keygen -t ed25519	Generate SSH key
ssh-copy-id	ssh-copy-id user@server	Install public key

A very useful pattern:

```
ssh user@server
```

Then:

```
uptime
free -h
df -h
```

You've already performed a basic server health check.

10. Services & systemd

Modern Linux servers commonly use `systemd`.

Command	Example	Use case
systemctl status	systemctl status nginx	Service status
systemctl start	sudo systemctl start nginx	Start service

<code>systemctl stop</code>	<code>sudo systemctl stop nginx</code>	Stop service
<code>systemctl restart</code>	<code>sudo systemctl restart nginx</code>	Restart
<code>systemctl reload</code>	<code>sudo systemctl reload nginx</code>	Reload config
<code>systemctl enable</code>	<code>sudo systemctl enable nginx</code>	Start at boot
<code>systemctl disable</code>	<code>sudo systemctl disable nginx</code>	Disable startup
<code>journalctl</code>	<code>journalctl -u nginx</code>	Service logs
<code>journalctl -f</code>	<code>journalctl -u nginx -f</code>	Follow logs

Example:

```
sudo systemctl status nginx
```

If broken:

```
sudo journalctl -u nginx -n 100
```

11. Environment Variables

Very important for Node.js and containers.

Command	Example	Use case
<code>env</code>	<code>env</code>	Show environment variables
<code>printenv</code>	<code>printenv NODE_ENV</code>	Read variable

export	export NODE_ENV=production	Set variable
unset	unset NODE_ENV	Remove variable
echo	echo \$NODE_ENV	Display variable

Example:

```
export NODE_ENV=production
export PORT=3000
```

Check:

```
echo $NODE_ENV
echo $PORT
```



12. Package Management



Ubuntu/Debian:

Command	Example	Use case
apt update	sudo apt update	Update package index
apt upgrade	sudo apt upgrade	Upgrade packages
apt install	sudo apt install nginx	Install package
apt remove	sudo apt remove nginx	Remove package
apt search	apt search nginx	Search packages
dpkg	dpkg -l	Installed packages



13. Archives & Compression

Command	Example	Use case
<code>tar</code>	<code>tar -czf app.tar.gz app/</code>	Create archive
<code>tar -xzf</code>	<code>tar -xzf app.tar.gz</code>	Extract archive
<code>gzip</code>	<code>gzip app.log</code>	Compress
<code>gunzip</code>	<code>gunzip app.log.gz</code>	Decompress
<code>zip</code>	<code>zip -r app.zip app/</code>	ZIP archive
<code>unzip</code>	<code>unzip app.zip</code>	Extract ZIP

Very common deployment operation:

```
tar -czf release.tar.gz dist package.json
```

Then:

```
scp release.tar.gz server:/tmp/
```

14. `find` — One of the Most Important Commands

Master this.

Requirement	Command
-------------	---------

Find file	<code>find . -name "app.log"</code>
Find all .log	<code>find . -name "*.log"</code>
Find directories	<code>find . -type d</code>
Find files	<code>find . -type f</code>
Files > 100 MB	<code>find . -type f -size +100M</code>
Files modified today	<code>find . -mtime 0</code>
Files older than 7 days	<code>find . -mtime +7</code>
Delete .tmp files	<code>find . -name "*.tmp" -delete</code>

Be careful with:

```
-delete
```

on production systems.

15. Pipes & Redirection

This is where Linux becomes extremely powerful.

Pipe

```
ps aux | grep node
```

Output of one command becomes input to another.

Redirect output

```
ls -lah > files.txt
```

Append

```
echo "deployment successful" >> deploy.log
```

Error redirection

```
command 2> error.log
```

Both stdout + stderr

```
command > output.log 2>&1
```

Discard output

```
command > /dev/null 2>&1
```

You should become very comfortable with:

```
|  
>  
>>  
2>  
2>&1
```

16. xargs

Very useful when working with large command outputs.

Example:

```
find . -name "*.log" | xargs ls -lh
```

Or:

```
cat services.txt | xargs -n1 docker restart
```

Understand what you're executing before using `xargs` with destructive commands.

17. Git + Linux

DevOps engineers constantly combine Git with Linux.

Command	Example	Use case
git clone	git clone repo.git	Clone repository
git status	git status	Working tree
git log	git log --oneline	Commit history
git branch	git branch	Branches
git diff	git diff	Changes
git pull	git pull	Update code
git checkout	git checkout main	Switch branch

For your banking platform:

```
git pull
pnpm install
pnpm build
docker compose up -d
```

```
docker compose ps
docker compose logs -f
```

That's a very realistic DevOps workflow.

18. Docker + Linux Commands

Since you're learning Docker, practice these together.

Command	Example	Use case
docker ps	docker ps	Running containers
docker ps -a	docker ps -a	All containers
docker images	docker images	Images
docker logs	docker logs api	Container logs
docker exec	docker exec -it api sh	Enter container
docker inspect	docker inspect api	Container details
docker stats	docker stats	Resource usage
docker port	docker port api	Port mappings
docker cp	docker cp api:/app/log.txt .	Copy files
docker network ls	docker network ls	Networks
docker volume ls	docker volume ls	Volumes

A very important debugging sequence:

```
docker ps
```

```
docker logs api
```

```
docker exec -it api sh
```

Inside:

```
ps  
ls -lah  
env
```

19. Process + Network Troubleshooting

Memorize this workflow.

Problem:

"My application isn't accessible."

Run:

```
ps aux | grep node
```

Then:

```
ss -tulpn
```

Then:

```
curl localhost:3000
```

Then:

```
curl localhost:3000/health
```

If Docker:

```
docker ps
```

```
docker logs api
```

```
docker port api
```

This is much more valuable than memorizing 500 commands.

20. Advanced Commands You Should Eventually Learn

Command	Example	Why learn it
lsof	<code>lsof -i :3000</code>	Find process using port
strace	<code>strace -p 1234</code>	Debug system calls
tcpdump	<code>sudo tcpdump -i any port 5432</code>	Network packet debugging
nc	<code>nc -vz localhost 5432</code>	Test TCP connectivity

openssl	openssl s_client - connect example.com:443	TLS debugging
dig	dig api.example.com	DNS debugging
journalctl	journalctl -xe	System/service troubleshooting
dmesg	dmesg tail	Kernel issues
vmstat	vmstat 1	System performance
iostat	iostat -xz 1	Disk I/O
sar	sar -n DEV 1	Historical/system metrics

Your Daily 30-Minute Linux Practice

Don't try to memorize everything above.

Do this **every day**:

5 min — Files

```
pwd
ls -lah
cd
mkdir
cp
mv
rm
find
```

5 min — Logs

```
cat  
less  
head  
tail  
tail -f  
grep  
awk  
sed
```

5 min — System

```
ps aux  
top  
free -h  
df -h  
du -sh  
uptime
```

5 min — Networking

```
ip addr  
ip route  
ss -tulpn  
curl  
ping  
dig  
lsof
```

5 min — Permissions

```
ls -l
```

```
chmod  
chown  
id  
sudo
```

5 min — DevOps

Use your own project:

```
git pull  
docker ps  
docker images  
docker logs  
docker exec  
docker stats  
docker network ls
```

☆ The 25 Commands I'd Make Muscle Memory

If you want to become a strong DevOps engineer, start with these:

```
pwd  
ls  
cd  
mkdir  
cp  
mv  
rm  
cat  
less  
tail  
grep  
find
```

```
ps
top
kill
df
du
free
chmod
chown
curl
ss
ssh
systemctl
journalctl
```

Then add:

```
awk
sed
xargs
lsof
dig
tcpdump
strace
rsync
tar
```

Most importantly

Don't learn Linux as a **command dictionary**.

Learn it as a **troubleshooting system**:

```
Application problem
    ↓
Process?
    ↓
ps / top
    ↓
```


Port?



ss / lsof



Network?



ping / curl / nc / dig



Logs?



tail / grep / journalctl



CPU / RAM?



top / free / vmstat



Disk?



df / du



Permissions?



ls -l / chmod / chown

That mindset is far more important for a **Senior DevOps / Platform Engineer** than knowing hundreds of commands.